PYTHON SOURCE CODE — SHOP INVENTORY ANALYSIS

Print appendix. The executable .py file is included separately in the project folder.

```
001: import pandas as pd
002: import numpy as np
003: import matplotlib
004: matplotlib.use("Agg")  # Save charts reliably on computers without a GUI
005: import matplotlib.pyplot as plt
006: from sklearn.model_selection import train_test_split
007: from sklearn.linear_model import LinearRegression, LogisticRegression
008: from sklearn.metrics import mean_squared_error, accuracy_score
009:
010: print("=" * 50)
011: print("SHOP INVENTORY DATA ANALYSIS & DASHBOARD")
012: print("=" * 50)
013:
014: file_name = "shop_data.csv"
015: df = pd.read_csv(file_name)
016:
017: print("\n[INFO] Initializing Data Cleaning...")
018: print(f"Original dataset shape: {df.shape} (Checking for Null Values)")
019: df_clean = df.dropna().copy()
020: print(f"Cleaned dataset shape:  {df_clean.shape} (Null Values Removed)")
021:
022: if len(df_clean) < 10:
023:     raise ValueError("At least 10 complete rows are recommended for this demonstration.")
024: if df_clean["Out_of_Stock"].nunique() < 2:
025:     raise ValueError("Out_of_Stock must contain both 0 and 1 for logistic regression.")
026:
027: print("\n" + "-" * 50)
028: print("MODEL 1: LINEAR REGRESSION")
029: print("-" * 50)
030: X_lin = df_clean[["Price", "Daily_Visitors"]]
031: y_lin = df_clean["Units_Sold"]
032: X_train_lin, X_test_lin, y_train_lin, y_test_lin = train_test_split(
033:     X_lin, y_lin, test_size=0.2, random_state=42
034: )
035: lin_model = LinearRegression()
036: lin_model.fit(X_train_lin, y_train_lin)
037: lin_predictions = lin_model.predict(X_test_lin)
038: mse = mean_squared_error(y_test_lin, lin_predictions)
039: print("[SUCCESS] Linear Regression model trained.")
040: print(f"[RESULT] Mean Squared Error: {mse:.2f}")
041:
042: print("\n" + "-" * 50)
043: print("MODEL 2: LOGISTIC REGRESSION")
044: print("-" * 50)
045: X_log = df_clean[["Units_Sold"]]
046: y_log = df_clean["Out_of_Stock"]
047: X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(
048:     X_log, y_log, test_size=0.2, random_state=42, stratify=y_log
049: )
050: log_model = LogisticRegression(max_iter=1000)
051: log_model.fit(X_train_log, y_train_log)
052: log_predictions = log_model.predict(X_test_log)
053: accuracy = accuracy_score(y_test_log, log_predictions)
054: print("[SUCCESS] Logistic Regression model trained.")
055: print(f"[RESULT] Classification Accuracy: {accuracy * 100:.2f}%")
056: print("=" * 50)
057:
058: # Four-chart dashboard
059: fig, axs = plt.subplots(2, 2, figsize=(14, 10))
060: fig.suptitle("Comprehensive Shop Data Analysis", fontsize=16, fontweight="bold")
061:
062: axs[0, 0].scatter(df_clean["Price"], df_clean["Units_Sold"], alpha=0.7)
```

```
063: axs[0, 0].set_title("Scatter Plot: Price vs. Units Sold")
064: axs[0, 0].set_xlabel("Price")
065: axs[0, 0].set_ylabel("Units Sold")
066: axs[0, 0].grid(True, linestyle="--", alpha=0.5)
067:
068: axs[0, 1].hist(df_clean["Units_Sold"], bins=15, edgecolor="black", alpha=0.75)
069: axs[0, 1].set_title("Histogram: Distribution of Units Sold")
070: axs[0, 1].set_xlabel("Units Sold")
071: axs[0, 1].set_ylabel("Frequency")
072: axs[0, 1].grid(True, linestyle="--", alpha=0.5)
073:
074: subset = df_clean.head(30)
075: axs[1, 0].plot(subset["Product_ID"], subset["Units_Sold"], marker="o", linewidth=1.5)
076: axs[1, 0].set_title("Line Graph: Units Sold for First 30 Products")
077: axs[1, 0].set_xlabel("Product ID")
078: axs[1, 0].set_ylabel("Units Sold")
079: axs[1, 0].grid(True, linestyle="--", alpha=0.5)
080:
081: columns = ["Price", "Daily_Visitors", "Units_Sold", "Out_of_Stock"]
082: corr = df_clean[columns].corr()
083: cax = axs[1, 1].imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
084: fig.colorbar(cax, ax=axs[1, 1])
085: axs[1, 1].set_xticks(np.arange(len(columns)))
086: axs[1, 1].set_yticks(np.arange(len(columns)))
087: axs[1, 1].set_xticklabels(columns, rotation=45, ha="right")
088: axs[1, 1].set_yticklabels(columns)
089: axs[1, 1].set_title("Heat Map: Feature Correlation")
090: for i in range(len(columns)):
091:     for j in range(len(columns)):
092:         axs[1, 1].text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
093:                        color="black" if abs(corr.iloc[i, j]) < 0.5 else "white")
094:
095: fig.tight_layout(h_pad=3, w_pad=2)
096: fig.subplots_adjust(top=0.90)
097: fig.savefig("dashboard_output.png", dpi=160, bbox_inches="tight")
098: print("[OUTPUT] Dashboard saved as dashboard_output.png")
099: print("[INFO] Run completed successfully.")
```